

Một số cấu trúc lưu trữ thường dùng để giải bài toán quy mô không gian

Long Chong

Nguồn gốc: Several common storage structures for solving space-scale problems

I0I China candidate team papers / 2000 / Long Chong.pdf

Mục lục

1	Dẫn nhập	2
2	Một vài cấu trúc dữ liệu thường dùng	2
2.1	Cấu trúc liên kết	2
2.2	Cấu trúc cây	3
2.3	Cấu trúc ma trận	5
3	Kết luận	9
A	Ghi chú	9
B	Phân tích chương trình	9
C	Tài liệu tham khảo	21

Từ khóa. Quy mô không gian; cấu trúc lưu trữ.

Tóm tắt

Các bài toán kiểu quy mô không gian là một điểm nóng trong các kỳ thi trong và ngoài nước những năm gần đây. Loại bài toán này đặt ra yêu cầu cao hơn đối với năng lực nắm vững và vận dụng nhiều cấu trúc lưu trữ dữ liệu của thí sinh. Bài viết đứng từ góc độ giải các bài toán kiểu quy mô không gian, phân tích sâu một vài đặc điểm mới mà một số cấu trúc lưu trữ dữ liệu thường dùng thể hiện trong loại bài toán đặc biệt này, đồng thời tổng kết ưu điểm và nhược điểm riêng của danh sách liên kết đơn, danh sách liên kết đôi, cấu trúc cây và cấu trúc ma trận. Đặc biệt, thông qua việc so sánh nhiều lời giải cho cùng một bài, bài viết cho thấy cấu trúc ma trận có tiềm năng rất lớn. Bài viết phân tích và tổng kết chi tiết ba đề thi trong nước và quốc tế; điều đó có ý nghĩa thực tiễn rõ rệt đối với thí sinh tham gia các kỳ thi trong nước hoặc quốc tế.

1 Dẫn nhập

Trong *Từ điển tiếng Hán hiện đại*, từ “quy mô” được giải thích là “phạm vi mà một sự vật nào đó bao gồm”. Trong thiết kế chương trình máy tính, ta có thể hiểu nó là chi phí mà chương trình cần khi chạy, xét trên không gian, thời gian, v.v. Nó chủ yếu gồm hai mặt: quy mô không gian và quy mô thời gian. Cả hai đều là các vấn đề thường xuyên phải xét khi thiết kế thuật toán và viết chương trình. Cấu trúc lưu trữ là cách biểu diễn trong máy tính của các phần tử dữ liệu và quan hệ giữa chúng; nó là những phương pháp lưu trữ được tổng kết để giải vấn đề quy mô không gian, hoặc gián tiếp giải vấn đề quy mô thời gian thông qua việc giải vấn đề quy mô không gian.

Những năm gần đây, trong các kỳ thi tin học đã xuất hiện một loại bài toán mới. Đặc điểm chung của chúng là dữ liệu vào rất lớn, có thể đạt 1 MB, thậm chí vài MB. Ta tạm gọi loại này là bài toán kiểu quy mô không gian. Như mọi người đều biết, khi lập trình bằng Turbo Pascal, ta chỉ dùng được bộ nhớ thường tối đa 640 KB; chỉ với lượng bộ nhớ hữu hạn ấy, hoàn toàn không thể lưu toàn bộ dữ liệu từng phần một. Điều này đặt ra yêu cầu cao hơn đối với cấu trúc lưu trữ dữ liệu và việc thiết kế thuật toán của chúng ta. Xem ghi chú 1..

Chương trình là sự kết hợp hữu cơ giữa thuật toán và cấu trúc dữ liệu. Bài viết này tập trung xuất phát từ góc độ cấu trúc dữ liệu, nhằm vào các bài toán quy mô dữ liệu lớn, để nghiên cứu lại tình hình vận dụng một số cấu trúc lưu trữ thường dùng và những đặc điểm mới mà chúng thể hiện.

2 Một vài cấu trúc dữ liệu thường dùng

Qua nghiên cứu kỹ các mô thức cấu trúc lưu trữ trong loại bài toán này ở các kỳ thi trong nước và quốc tế, tôi thấy rằng các cấu trúc lưu trữ thường dùng để giải chúng có ba loại: cấu trúc liên kết, cấu trúc cây và cấu trúc ma trận. Dưới đây tôi lần lượt phân tích tình hình ứng dụng của ba cấu trúc ấy trong loại bài toán này, đồng thời so sánh ưu điểm và nhược điểm riêng của chúng.

2.1 Cấu trúc liên kết

Trong cấu trúc liên kết, hai kiểu thường dùng là danh sách liên kết đơn và danh sách liên kết đôi. Ta bắt đầu từ một ví dụ đơn giản.

Bài toán 1: Tuyển tham quan tốt nhất, đề 1 vòng 2 NOI 1997. Một khu du lịch có đường phố dạng mạng. Các đường đông-tây là phố du lịch, chỉ được đi từ tây sang đông, và có một điểm số nhất định. Các đường nam-bắc là đường rợp bóng cây, có thể đi từ nam lên bắc hoặc từ bắc xuống nam, và không có điểm số. Cần bắt đầu tham quan từ một giao lộ và kết thúc ở một giao lộ khác sao cho tổng điểm của các phố đã đi qua là lớn nhất. Trong đó $1 \leq$ số phố du lịch ≤ 100 , $1 \leq$ số đường rợp bóng cây ≤ 20001 , và $-100 \leq$ điểm số ≤ 100 .

Phân tích. Thoạt nhìn, quy mô của bài này rất đáng sợ: $100 \times 20001 = 2000100$. Ta không thể lưu một quy mô khổng lồ như vậy. Nhưng suy nghĩ kỹ hơn, vì chỉ được đi từ tây sang đông, mỗi cột dọc nhiều nhất chỉ đi qua một lần. Đối với các phố du lịch trong cùng một cột dọc, ta có thể dùng đường rợp bóng cây để tự do đi tới bất kỳ phố du lịch nào. Để đạt yêu cầu tối ưu của đề, ta chỉ cần chọn con đường có điểm số lớn nhất. Vì vậy, khi lưu trữ, ta chỉ cần ghi lại điểm lớn nhất trong mỗi cột dọc; cấu trúc lưu trữ từ hai chiều biến thành một chiều, dùng 20001 phần tử kiểu `short int`. Phân tích thêm, ta thấy: nếu $A \rightarrow B$ là một tuyến tham quan tối ưu, và C là một điểm nằm giữa A và B , thì điểm số trên tuyến con $A \rightarrow C$ chắc chắn không âm; nếu không, điểm số của $C \rightarrow B$ sẽ lớn hơn điểm số của $A \rightarrow B$.

Định nghĩa P_i là tổng điểm của đường đi tối ưu kết thúc tại cột i , và A_i là điểm lớn nhất của cột dọc thứ i . Ta có phương trình chuyển trạng thái:

$$P_i = \begin{cases} P_{i-1} + A_i, & \text{nếu } P_{i-1} + A_i \text{ không âm,} \\ 0, & \text{nếu } P_{i-1} + A_i \text{ âm.} \end{cases}$$

Điều kiện biên là $P_0 = 0$. Mục tiêu là tìm $\max\{P_i\}$.

Thông qua chuyển hóa này, ta có thể vừa đọc tệp vừa chuyển dữ liệu hai chiều thành dữ liệu một chiều. Sau đó tính từ P_1 ; mỗi lần tính P_i chỉ cần một phép xét, rồi lập tức chuyển sang P_{i+1} , cho tới khi xử lý hết mọi phần tử. Xem chương trình 1. Ta có thể biểu diễn cấu trúc ấy như sau:

a1 -> a2 -> a3 -> ... -> an

Hình 1-1: danh sách liên kết đơn.

Đây chính là danh sách liên kết đơn mà ta quen thuộc. Trong bài toán, nó thể hiện như sau: để xử lý phần tử hiện tại, ta không cần quay lại tìm các phần tử phía trước, vì ảnh hưởng của các phần tử phía trước lên nó là hữu hạn và có thể biểu diễn trực tiếp bằng vài biến. Với biểu hiện ấy, ta có thể tổng kết các đặc điểm của danh sách liên kết đơn khi giải bài toán dữ liệu quy mô không gian:

1. Hiệu suất cao. Trong tình huống bình thường, chương trình viết bằng cấu trúc này thường không gặp vấn đề về thời gian. Lý do rất đơn giản: mỗi phần tử chỉ cần vài phép tính rất hữu hạn, nên độ phức tạp thấp. Xem ghi chú 2..
2. Do cấu trúc danh sách liên kết đơn quá đơn giản, trong thiết kế thuật toán cụ thể nó phản ánh quan hệ logic giữa các phần tử không đủ rõ, vì giữa các phần tử chỉ có một cạnh một chiều nối các phần tử kề nhau. Do đó, danh sách liên kết đơn có hạn chế lớn khi giải bài; đối với bài khó hơn, rất khó dùng nó để hiện thực. Tuy vậy, vì đây là một cấu trúc hiệu quả như thế, nó vẫn là một hướng mà ta nên cố gắng đạt tới khi thiết kế chương trình.

Danh sách liên kết đôi trong cấu trúc liên kết cũng là một cấu trúc dữ liệu được dùng thường xuyên trong lập trình. Nó có thể biểu diễn như sau:

... <-> a_{i-1} <-> a_i <-> a_{i+1} <-> ...

Hình 1-2: danh sách liên kết đôi.

Trong đó, ngoài phần tử được lưu, mỗi ô còn có một con trỏ tiền nhiệm và một con trỏ kế nhiệm. Điều này ở mức độ nhất định khắc phục khuyết điểm của danh sách liên kết đơn là biểu diễn quan hệ giữa các phần tử quá đơn giản. Nhưng khi dùng danh sách liên kết đôi, cần chú ý hết sức khống chế độ sâu tìm ngược, vì số phần tử trong dữ liệu vào của loại bài này rất lớn; xử lý không thỏa đáng sẽ làm quy mô thời gian của chương trình tăng mạnh. Vì vậy danh sách liên kết đôi cũng có hạn chế nhất định. Để phân tích rõ hơn đặc điểm của nó, ta sẽ tiếp tục bàn ở phần sau.

2.2 Cấu trúc cây

Cây là cấu trúc dữ liệu quen thuộc với chúng ta. Đại diện tiêu biểu nhất là cây nhị phân, một cấu trúc phi tuyến đặc biệt, được ứng dụng rộng rãi trong lập trình.

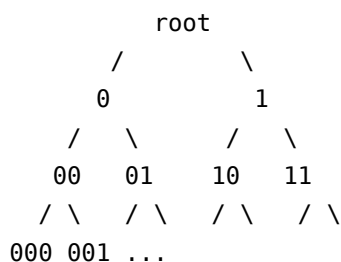
Bài toán 2: Contact, đề 1 ngày 1 IOI 1998. Từ một tệp tạo bởi xâu 0/1, tìm N tần suất khác nhau lớn nhất của các xâu con có độ dài nằm giữa A và B . Các xâu con có thể chồng lấn nhau. Kết quả xuất phải sắp theo thứ tự giảm dần của tần suất xuất hiện; các xâu cùng tần suất sắp theo độ dài giảm dần; các xâu có cùng tần suất và cùng độ dài thì sắp theo giá trị tương ứng giảm dần. Trong đó $0 < A \leq B \leq 12$, $0 < N \leq 20$, và tệp vào có thể đạt 2 MB.

Phân tích. Bài này yêu cầu hoàn thành hai thao tác:

1. Tìm mọi xâu con thỏa điều kiện và thống kê tần suất xuất hiện của từng xâu.
2. Sắp mọi xâu con khác nhau theo yêu cầu rồi xuất ra.

Trong đó bước 1 là then chốt. Ta bắt đầu nghiên cứu từ ví dụ cụ thể. Xâu độ dài 1 chỉ có hai khả năng, “0” và “1”. Xâu độ dài 2 có bốn khả năng: “00”, “01”, “10”, “11”; chúng có thể xem là được tạo ra bằng cách thêm “0” hoặc “1” vào sau một xâu con độ dài 1. Tương tự, xâu độ dài 3 lại có thể xem là được tạo ra bằng cách thêm “0” hoặc “1” vào sau một xâu con độ dài 2; chẳng hạn “101” được tạo ra bằng cách thêm “1” vào sau “10”. Về sau cũng có thể suy ra như vậy. Quan hệ từng tầng tiến sâu ấy khiến ta nghĩ tới cây.

Dưới đây là một cây nhị phân, nút trên cùng là gốc. Ta định nghĩa nhánh trái của mỗi nút là 0, nhánh phải là 1. Khi đó, một xâu con có thể tương ứng một-một với một đường đi trong cây nhị phân:



Hình 2-1: đường đi trong cây nhị phân ứng với xâu 0/1.

Mỗi điểm trong cây được gán một trọng số, biểu thị tần suất xuất hiện trong tệp của xâu con tương ứng với điểm ấy. Khi đó ta có thể nhận được một số quy luật cộng dồn. Ví dụ, khi $A = 1$, $B = 3$, giả sử một trạng thái trung gian tương ứng với cây nhị phân nào đó. Nếu hiện đang đọc tới xâu “011”, các xâu con bắt đầu bằng “0” và có độ dài từ 1 tới 3 là ba xâu: “0”, “01”, “011”. Khi thống kê, cần tăng tần suất của ba xâu này lên 1. Trên hình, có thể trực quan thấy thao tác ấy tương đương với việc tăng tần suất của các nút trên đường đi $0 \rightarrow 01 \rightarrow 011$ thêm 1.

Vì số nút của cây nhị phân tương ứng rất ít, tối đa là $2^{13} - 1 = 8191$, hoàn toàn có thể duyệt nhiều lần. Không khó để tìm ra N xâu con có tần suất lớn nhất, rồi xuất theo thứ tự từ lớn đến nhỏ. Xem chương trình 2.

Từ bài này ta thấy, so với danh sách liên kết đơn, trong cấu trúc cây mỗi nút có liên hệ một-nhiều với các nút khác. Điều đó có lợi hơn cho việc xử lý dữ liệu. Đặc biệt, khi xử lý bài toán quy mô không gian, nó thể hiện các đặc điểm sau:

1. Tính quy luật mạnh. Nếu quan hệ giữa nút thế hệ trước và nút con được xây dựng tốt, ta có thể sắp xếp lượng phần tử khổng lồ rất trật tự, rồi theo một thứ tự nhất định đi sâu từng tầng để xử lý phần tử hiện tại.
2. Tính thao tác mạnh. Ta khá thành thạo nhiều phép thao tác trên cây. Khi giải bài, chỉ cần dựa vào đặc điểm đề để xây dựng tốt các quan hệ của cây, những thao tác khác như tìm kiếm và in ra sẽ được giải quyết thuận lợi.

2.3 Cấu trúc ma trận

Thông thường ta dùng mảng hai chiều để biểu diễn cấu trúc ma trận. Nó có hai hướng x và y . Trong thao tác thực tế, cách biểu diễn thường dùng là: trục x biểu thị các phần tử dữ liệu, trục y biểu thị các điều kiện trạng thái khác nhau của phần tử. Giá trị cụ thể trong mảng biểu thị biểu hiện của phần tử dưới điều kiện trạng thái hiện tại, thường bằng một giá trị số. Có thể hình dung như sau:

điều kiện trạng thái y	$a_{1,n}$	$a_{2,n}$	$\cdots$	$a_{m,n}$
	$\vdots$	$\vdots$		$\vdots$
	$a_{1,2}$	$a_{2,2}$	$\cdots$	$a_{m,2}$
	$a_{1,1}$	$a_{2,1}$	$\cdots$	$a_{m,1}$
	x : phần tử dữ liệu			

Bài toán 2 tiếp: *Contact*, đề 1 ngày 1 IOI 1998.

Phân tích tiếp. Tiếp tục phân tích bài này, ta còn thấy rằng mỗi xâu con chỉ gồm 0 và 1 cũng có thể xem là một số nhị phân rồi chuyển thành số thập phân. Tuy nhiên, mỗi số thập phân không tương ứng một-một với xâu 0/1; ví dụ “01”, “010” và “0010” đều tương ứng với số thập phân 2. Vì sao? Trong nhị phân, 10, 010 và 0010 biểu diễn cùng một số, nhưng các xâu “10”, “010” và “0010” lại không phải cùng một xâu, vì lúc này ký tự “0” không biểu thị “không có”, mà là một ký tự; giữa chúng có khác biệt về độ dài. Để phân biệt các xâu có độ dài khác nhau nhưng sau khi chuyển sang số thập phân lại bằng nhau, ta đặt thêm tọa độ “độ dài”. Như vậy, mảng một chiều biến thành ma trận hai chiều, có thể biểu diễn đơn giản như sau:

độ dài xâu 01, trục Y	tần suất xuất hiện
X : số thập phân tương ứng với xâu 01	

Từ bảng, ta có thể tìm N xâu có tần suất lớn nhất, rồi xuất theo yêu cầu. Xem chương trình 3.

So sánh hai cách, vì chương trình 3 dùng thao tác mảng, còn chương trình 2 phải đệ quy nhiều lần, cách sau ngắn gọn hơn và cũng nhanh hơn. Bảng dưới đây cho thấy thời gian chạy; hai bộ dữ liệu đầu là hai bộ lớn hơn trong kỳ thi quốc tế, hai bộ sau tự tạo và hơi lớn hơn 2 MB. Xem ghi chú 3..

	Dữ liệu 1	Dữ liệu 2	Dữ liệu 3	Dữ liệu 4
Chương trình 2	1.4 s	2.3 s	5.8 s	6.0 s
Chương trình 3	0.5 s	1.2 s	3.9 s	3.5 s

Do cấu trúc ma trận kết hợp phép tính nhị phân trong toán học, hiệu suất chương trình lại tăng thêm. Có phải cấu trúc ma trận có tiềm năng lớn không? Thật ra, cấu trúc ma trận còn có thể kết hợp với nhiều mô hình toán học khác, trong đó có cả quy hoạch động. Khi dùng nó để giải một số bài khó, ta thường nhận được hiệu quả bất ngờ. Bây giờ ta thử phân tích thêm một bài.

Bài toán 3: Bài toán mã ẩn *codes*, đề IOI 1999. Chi tiết đề xem đề thi IOI 1999.

Phân tích. Bài này chủ yếu cần giải hai trọng điểm:

1. Tìm dãy ẩn phải nhỏ nhất.
2. Sau khi tìm được dãy ẩn phải nhỏ nhất, tìm các mã ẩn không phủ nhau sao cho tổng độ dài các mã từ được chứa là lớn nhất.

Vấn đề thứ hai tương đối đơn giản. Định nghĩa P_j là tổng độ dài mã lớn nhất của các mã ẩn không phủ nhau từ đầu văn bản tới chữ cái thứ j . Khi chưa tìm thấy mã ẩn kết thúc tại j , $P_j = 0$. Nếu hiện tìm thấy một mã ẩn bắt đầu tại i , kết thúc tại j , và khớp với mã từ có độ dài L , ta có thể viết phương trình:

$$P_j = \max\{P_j, P_k + L\}, \quad 1 \leq k < j, \quad P_0 = 0.$$

Mục tiêu là

$$\max\{P_j\}, \quad 1 \leq j \leq \text{độ dài văn bản}.$$

Nhưng đề cho độ dài văn bản tới một triệu, không phải sao? Thật ra, dãy ẩn phải nhỏ nhất không vượt quá 10000 dãy; vì vậy số lượng khác nhau nhiều nhất cũng chỉ là một vạn. Khi lập trình thực tế, ta không cần lấy j làm chỉ số trực tiếp trên toàn văn bản.

Bây giờ ta tập trung giải vấn đề dãy ẩn phải nhỏ nhất. Trên thực tế, ta còn phải tìm dãy ẩn phải nhỏ nhất ngắn nhất. Ví dụ, có hai mã từ TUN, NIR và một văn bản TUNNIR. Theo định nghĩa, NNIR cũng là dãy ẩn phải nhỏ nhất; nhưng nếu tính theo nó, ta chỉ có thể tìm được TUN hoặc NNIR. Vì vậy, khi thiết kế thuật toán, ta phải tính theo dãy ngắn nhất, tức NIR.

Tiếp tục suy nghĩ, tôi nghĩ ra hai phương pháp. Chúng dùng các cấu trúc lưu trữ khác nhau; dưới đây đưa ra để so sánh và phân tích.

Phương pháp A. Khi đọc tới một ký tự, ta có thể dùng một mảng P_i . Nó biểu thị mã từ thứ i , tính tới ký tự hiện tại, đã khớp tới độ dài lớn nhất nào, với điều kiện chưa khớp xong. Ví dụ mã từ là ALL, văn bản là AAALAL; ta có thể hiện thực như sau:

```
AAALAL
P -> 111223
```

Như vậy, tới ký tự cuối cùng ta đã có một mã ẩn. Nhưng ta vẫn chưa thể xác định nó có phải dãy ẩn trái nhỏ nhất ngắn nhất hay không, cũng chưa thể xác định độ dài của nó có nằm trong 1000 hay không. Vì vậy phải quay ngược: tức là tìm từ ký tự cuối cùng, lần lượt tìm từng ký tự từ cuối về đầu cho tới khi tìm được ký tự đầu tiên. Tối đa chỉ quét ngược 999 ký tự; nếu vẫn chưa tìm xong thì chứng tỏ mã ẩn vượt quá 1000.

```
AAALAL
1233 <-
```

Như vậy thứ ta tìm được là dãy ẩn phải nhỏ nhất ngắn nhất. Qua phân tích này, ta biết phương pháp A dùng cấu trúc lưu trữ danh sách liên kết đôi:

```
... 1 -- 2 -- 3 -- ... -- 1000 -- 1001
      ^
      phan tu hien tai

<----- duong quet nguc lon nhat ----->
```

Hình 3-3

Đồng thời, ta cũng cần chú ý rằng trong trường hợp xấu nhất, độ sâu quét ngược là 1000. Ở trên ta đã phân tích rằng khi quét ngược bằng danh sách liên kết đôi thì độ sâu không được quá lớn. Tuy nhiên, bài này có mặt đặc biệt của nó: số dãy ẩn phải nhỏ nhất không vượt quá 10000, nên số lần quay ngược sẽ không nhiều. Về mặt lý thuyết, thuật toán khả thi. Nhưng đứng từ góc độ tinh chỉnh để tốt hơn, ta vẫn nên tiếp tục tìm một cấu trúc thích hợp hơn với bài này.

Phương pháp B. Tuy cấu trúc liên kết đôi của phương pháp A chưa thật lý tưởng, nó đã gợi ý cho ta khá nhiều. Điều quan trọng nhất là nó chú ý tới việc cần ghi lại vị trí đang khớp, nhờ đó việc tìm kiếm có một hướng nhất định; điều này cần được khẳng định. Tuy vậy, phương pháp A lại bỏ qua việc ghi nhận vị trí bắt đầu của dãy ẩn. Để tìm vị trí bắt đầu của dãy ẩn hiện tại, nó phải quét ngược mù quáng, gây lãng phí thời gian rất lớn. Tự nhiên ta sẽ nghĩ: liệu có thể ghi lại điểm bắt đầu ngay trong quá trình khớp hay không? Có lẽ điều đó làm được.

Vậy, khi đã ghi lại vị trí bắt đầu, có còn cần ghi lại vị trí hiện đang khớp trong mã từ giống như phương pháp A không? Ta chú ý tới ví dụ AAALA dùng khi giới thiệu phương pháp A. Nếu thêm một L ở phía sau, thành AAALALL, thì văn bản này có hai mã ẩn phải nhỏ nhất ngắn nhất AAALALL. Trong đó ký tự L thứ hai có hai thân phận: trong ALAL nó biểu thị ký tự thứ ba của mã từ, còn trong ALL nó biểu thị ký tự thứ hai của mã từ. Như vậy, do vị trí bắt đầu của mã ẩn tương ứng khác nhau, cùng một ký tự trong cùng một văn bản lại có trạng thái tương ứng khác nhau trong cùng một mã từ. Đây cũng là điểm mà phương pháp A chưa xử lý tốt.

Vậy cần xử lý thế nào? Trong phần phân tích cấu trúc ma trận, ta đã giới thiệu rằng hướng trục Y trong ma trận thường dùng để biểu thị các trạng thái khác nhau của phần tử. Ta cũng có thể định nghĩa như sau: trục X là số hiệu các mã từ; trục Y là vị trí đã khớp tới trong từng mã, tức chữ cái thứ mấy; $P_{x,y}$ biểu thị vị trí bắt đầu trong toàn văn bản của dãy ẩn phải nhỏ nhất ngắn nhất, gần ký tự hiện đang đọc nhất, có chứa y chữ cái đầu của mã từ thứ x . Chú ý rằng lúc này trục X đã thay đổi theo ý đề, không còn biểu thị ký tự hiện đang đọc. Đồng thời, ta cũng thấy rõ rằng các phần tử trong cùng một cột của ma trận có quan hệ mật thiết, còn các phần tử trong cùng một hàng không liên hệ nhiều. Có thể biểu diễn:

vị trí khớp Y	$P_{1,n}$	$P_{2,n}$	$\cdots$	$P_{m,n}$
	$\vdots$	$\vdots$		$\vdots$
	$P_{1,2}$	$P_{2,2}$	$\cdots$	$P_{m,2}$
	$P_{1,1}$	$P_{2,1}$	$\cdots$	$P_{m,1}$
	X : số hiệu các mã từ			

Khi đã có cấu trúc lưu trữ, ta có thể thử suy ra thuật toán.

Định nghĩa $P_{i,j}$ là vị trí bắt đầu trong toàn văn bản của dãy ẩn phải nhỏ nhất ngắn nhất, gần ký tự hiện đang đọc nhất, có chứa j chữ cái đầu của mã từ thứ i . Khi chưa tìm thấy thì $P_{i,j} = 0$. Trong đó $1 < i \leq N$, $1 \leq j \leq \text{Length}(i)$, và $\text{Length}(i)$ biểu thị độ dài mã từ thứ i .

Theo định nghĩa, giá trị $P_{i,j}$ của mã từ thứ i có thể do giá trị trước nó $P_{i,j-1}$ quyết định, với điều kiện là các chữ cái đã đọc trong văn bản đã khớp với $j - 1$ chữ cái đầu của mã từ này, đồng thời chữ cái hiện đang đọc lại bằng chữ cái thứ j của mã từ. Khi đó có thể sửa giá trị $P_{i,j}$.

Giả sử $A_{i,j}$ biểu thị chữ cái thứ j của mã từ thứ i ($1 \leq i \leq N$, $1 \leq j \leq \text{Length}(i)$). NowChar biểu thị chữ cái hiện đọc trong văn bản, và NowNum biểu thị số thứ tự của chữ cái văn bản hiện đọc.

Nếu thỏa các quan hệ sau:

1. $A_{i,j} = \text{NowChar}$, tức chữ cái hiện đọc bằng chữ cái thứ j của mã từ thứ i ;
2. $j = 1$, tức đây là chữ cái đầu tiên của mã từ; hoặc
3. $j > 1$, tức không phải chữ cái đầu tiên của mã từ, đồng thời $P_{i,j-1} > 0$, tức các chữ cái đã đọc trong văn bản đã khớp với $j - 1$ chữ cái đầu của mã từ này, và $\text{NowNum} - P_{i,j-1} + 1 \leq 1000$, tức độ dài dãy ẩn không vượt quá 1000.

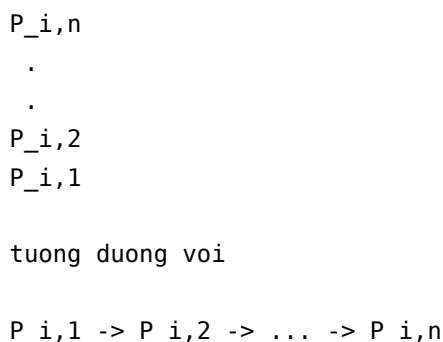
thì các công thức sau đúng:

$$P_{i,j} = \begin{cases} \text{NowNum}, & j = 1, \\ P_{i,j-1}, & j > 1. \end{cases}$$

Khi $P_{i, \text{Length}(i)} \neq 0$, tức khi j suy tới $\text{Length}(i)$, ta đã tìm được một “dãy ẩn phải nhỏ nhất” hợp yêu cầu, với số hiệu i , vị trí bắt đầu $P_{i, \text{Length}(i)}$ và vị trí kết thúc NowNum .

Đồng thời, khi đọc một ký tự từ văn bản, nếu muốn biết nó nằm ở vị trí nào trong từng mã từ, ta có thể lần lượt tìm trong tất cả mã từ. Nhưng như vậy hiệu suất không cao. Xét tổng số ký tự của mọi mã từ nhiều nhất chỉ là 100×100 , ta có thể phân loại theo chữ cái và ghi lại số hiệu các mã từ chứa chữ cái đó; nhờ vậy việc tìm kiếm sẽ nhanh hơn rất nhiều.

Trong quá trình suy diễn, ta phát hiện rằng trong bất kỳ quá trình khớp mã nào, ta luôn bắt đầu khớp từ ký tự thứ nhất; khi ký tự thứ nhất đã khớp, mới xét ký tự thứ hai, v.v. Do đó dễ thấy quan hệ giữa các phần tử trong mỗi cột thực chất là cấu trúc danh sách liên kết đơn:



Hình 3-5

Ở trên ta cũng đã nói rằng dùng danh sách liên kết đơn thường có hiệu suất rất cao. Phương pháp B dùng ma trận tạo bởi các danh sách liên kết đơn, suy ra công thức truy hồi quy hoạch động và loại bỏ độ phức tạp của việc quét ngược. Nó đáng để ta viết lại một chương trình. Xem chương trình 4. Kết quả chạy cũng nằm trong dự đoán: phương pháp B tốt hơn hẳn phương pháp A. Dưới đây là thời gian chạy của hai phương pháp trên các bộ dữ liệu tương ứng; ba bộ đầu là ba bộ dữ liệu cuối của kỳ thi quốc tế.

	Dữ liệu 1	Dữ liệu 2	Dữ liệu 3	Dữ liệu 4
Phương pháp A	3.4 s	2.8 s	4.0 s	9.9 s
Phương pháp B	0.5 s	0.3 s	1.1 s	5.1 s

Dựa trên hai ví dụ trên, ta đã có hiểu biết sâu hơn về cấu trúc ma trận. Vậy vì sao dùng cấu trúc ma trận để giải bài toán quy mô không gian lại có tiềm năng lớn? Trong phần phân tích bài toán, ta đã thấy khá nhiều điểm; nay tổng kết như sau:

1. Cấu trúc ma trận phát huy ưu điểm của cấu trúc liên kết trong việc biểu diễn trạng thái phần tử một cách đơn giản, rõ ràng. Nó dùng thao tác mảng, mà thao tác mảng thông thường cũng nhanh như danh sách liên kết.
2. Cấu trúc ma trận cũng phản ánh tốt tư tưởng của cấu trúc cây trong việc biểu diễn rõ quan hệ giữa các phần tử, đồng thời có một số ưu điểm mà cấu trúc cây không có. Thứ nhất, tìm theo tọa độ (X, Y) thuận tiện hơn rất nhiều so với duyệt cây. Thứ hai, ý nghĩa của hai trục X, Y có thể được định nghĩa kết hợp tốt hơn với phương pháp toán học. Như vậy, dùng cấu trúc ma trận đem lại không gian suy nghĩ rộng hơn.
3. Cấu trúc ma trận có thể dùng phương pháp tổng hợp để lồng danh sách liên kết và các cấu trúc khác vào bên trong, qua đó tăng tính linh hoạt khi giải bài. Điểm này rất quan trọng.

Vì vậy, khi thiết kế cấu trúc lưu trữ và thuật toán, không thể xem nhẹ cấu trúc ma trận. Tuy nhiên,

cũng không thể nói cấu trúc ma trận là vạn năng, và cũng không thể xem nhẹ việc vận dụng các cấu trúc khác trong giải bài.

3 Kết luận

Dùng một chương trình nhỏ để giải bài xử lý dữ liệu vài MB quả thật không dễ; viết chương trình giải các bài xử lý dữ liệu lớn trong các kỳ thi trong nước và quốc tế, lại còn có điều kiện ẩn, càng không dễ. Tuy nhiên, qua phân tích ở trên, ta có thể thấy loại bài này vẫn có quy luật nhất định. Trong luyện tập thường ngày, ta nên không ngừng tinh chỉnh các thuật toán liên quan tới cấu trúc dữ liệu; không nên chỉ bó hẹp trong các thuật toán xử lý thông thường, mà còn phải nghiên cứu phương pháp xử lý đặc thù của từng cấu trúc lưu trữ trong một số bài toán đặc biệt. Đồng thời, ta phải tổng kết nhiều hơn ưu điểm và nhược điểm mà mỗi cấu trúc thể hiện trong các tình huống khác nhau, bồi dưỡng năng lực phán đoán, phân tích và cân nhắc được mất. Như vậy, tư duy của ta mới rộng mở hơn và hiệu quả lập trình mới tốt hơn.

Cuối cùng, ta tổng kết lại đặc điểm của một số cấu trúc lưu trữ thường dùng để giải bài toán quy mô không gian. Các nhận xét đều nói về tình huống thông thường.

Loại	Dạng	Không gian	Tốc độ	Liên hệ phần tử	Cách xử lý trong bài quy mô không gian
Liên kết	Đơn	Ít	Nhanh	Ít	Truy hồi trực tiếp một bước, không cần quét ngược
Liên kết	Đôi	Ít	Chậm hơn	Nhiều	Quay lại xem thông tin liên quan tới phần tử hiện đọc
Cây		Nhiều	Trung bình	Nhiều	Cụ thể hóa liên hệ giữa các phần tử, thường dùng đệ quy
Ma trận		Nhiều	Khá nhanh	Rất nhiều	Truy hồi, kể cả quy hoạch động; khi cần thì quét ngược nhất định

A Ghi chú

1. Vì giá trị N rất lớn, mọi thủ pháp gây trễ khi xử lý từng phần tử dữ liệu đều có thể tích lũy và làm chương trình chậm bất thường. Vì vậy, ta nên tinh chỉnh các thuật toán thao tác dữ liệu thường ngày.
2. Ở đây không dùng ký hiệu $O(N^k)$ thường gặp để đo độ phức tạp thuật toán, vì quy mô dữ liệu, tức giá trị N , rất lớn. Độ phức tạp thời gian $O(N^2)$ hiển nhiên không thể chịu được. Quan trọng hơn, ngay cả $O(kN)$, nếu k rất lớn, chương trình cũng không thể vượt qua trong thời gian quy định.
3. Mọi chỉ số chạy đều đo trên máy Pentium III 450 MHz.

B Phân tích chương trình

Bốn chương trình sau đều dùng tên tệp vào, tệp ra và định dạng theo yêu cầu của đề gốc.

Chương trình 1

```
{ $A+, B-, D+, E+, F-, G-, I+, L+, N-, O-, P-, Q-, R-, S-, T-, V+, X+ }
{ $M 65520, 0, 655360 }
```

```
var m{pho du lich}, n{duong rop bong cay}:integer;
    a{diem tham quan hien tai, dung cho truy hoi},
```

```

b{diem tham quan lon nhat}:longint;
t:array[1..20001] of integer;{luu gia tri lon nhat cua tung cot}

procedure input;{nhap du lieu, luu gia tri lon nhat cua tung cot}
var f:text;
    i,j,k:integer;
begin
    assign(f,'input.txt'); reset(f);
    read(f,m,n); dec(n);
    for i:=1 to n do t[i]:=-maxint;{dat nho nhat de bat dau tim}
    for i:=1 to m do
        for j:=1 to n do
            begin
                read(f,k);
                if k>t[j] then t[j]:=k;{moi cot chi giu gia tri lon nhat}
            end;
        close(f);
    end;

procedure main;{truy hoi tinh ket qua}
var i,j:integer;
begin
    a:=0; b:=0;
    for i:=1 to n do
        begin
            inc(a,longint(t[i]));{cong don a}
            if a<0 then a:=0;{neu a am, bo het trang thai truoc, dat ve 0}
            if a>b then b:=a;{neu a tot hon b, lay a lam gia tri toi uu hien tai}
        end;
    end;

procedure print;{in ket qua}
var f:text;
begin
    assign(f,'output.txt');
    rewrite(f);
    writeln(f,b);
    close(f);
end;

begin
    input;{nhap du lieu, luu gia tri lon nhat cua tung cot}
    main;{truy hoi tinh ket qua}
    print;{in ket qua}
end.

```

Chương trình 2

```
{ $A+, B-, D+, E+, F-, G-, I+, L+, N-, O-, P-, Q-, R-, S-, T-, V+, X+ }  
{ $M 65520, 0, 655360 }
```

```
const inputfile = 'contact.in';  
  outfile = 'contact.out';  
  tc:array[1..13] of integer=  
    (1,2,4,8,16,32,64,128,256,512,1024,2048,4096);  
  {cac so nhi phan tuong ung la 1,10,100,..., tien cho lay bit}  
  tz:array['0'..'1'] of byte=(0,1);{chuyen ky tu thanh so}  
  
type r=^rr;  
  rr=record  
    zero{huong di khi ky tu hien tai la 0},  
    one{huong di khi ky tu hien tai la 1}:r;  
    num{tan suat cua xau con ung voi duong di hien tai}:longint;  
  end;  
ar=string[12];{phuong an xau con cu the}  
  
var t:array[1..21] of longint;{20 tan suat lon nhat, them o 21 de sap}  
  tch:array[1..1024] of char;{bo dem}  
  tree:r;{nut goc cua cay trang thai}  
  inf,outf:text;  
  a,b,n{bien vao cua de},long{do dai xau con hop le hien tai},  
  now{so tan suat khac nhau hien co khi sap},  
  which{vi tri tan suat can tim de xuat},  
  where{tim o tang nao}:byte;  
  yu{moc cat do dai hop le},m{bieu dien cua xau con hop le}:integer;  
  
procedure maketree(floor:byte; var p:r);{sinh cay rong, goc o tang 0}  
begin  
  p^.num:=0;{tong ban dau la 0}  
  
  if floor=b{da sinh den nut la} then  
  begin  
    p^.zero:=nil;  
    p^.one:=nil;{hai con tro trai phai deu rong}  
  end  
else  
  begin  
    new(p^.zero);{sinh cac nut con}  
    maketree(floor+1,p^.zero);  
    new(p^.one);  
    maketree(floor+1,p^.one);  
  end;  
end;
```

```

procedure init;{khởi tạo chương trình}
begin
  readln(inf,a,b,n);{nhập a,b,n để dùng khi lập cây}
  new(tree);{đặt nút gốc của cây}
  maketree(0,tree);{sinh cây rỗng}
end;

procedure add(floor:byte; var p:r);{cộng đơn theo từng tầng}
begin
  inc(p^.num);{tăng tầng}
  if floor<long then{kiểm tra đã đi đến cuối chưa}
    if ((m and tc[long-floor])=0){xét bit hiện tại để chọn đường đi}
      then add(floor+1,p^.zero)
      else add(floor+1,p^.one);
end;

procedure main;{đọc dữ liệu, cộng số lần xuất hiện của xâu con}
var ch:char;{ký tự hiện đọc}
    i:byte;{biên lặp}
begin
  long:=0;{chưa đọc dữ liệu, độ dài hợp lệ bằng 0}
  m:=0;{xâu hợp lệ ban đầu rỗng}
  yu:=tc[b+1];{ví dụ độ dài hợp lệ 2 thì lấy phần dư theo 4}
  repeat
    read(inf,ch);

    if ch='2' then{đã đọc hết tệp}
      begin
        for i:=long downto 1 do{xử lý các xâu con con lại có độ dài dưới b}
          begin
            add(0,tree);
            dec(long);
          end;
        exit;
      end;

    m:=(m*2+tc[ch]) mod yu;{sinh xâu con hợp lệ hiện tại}
    inc(long);
    if long=b then{nếu độ dài bằng b thì cộng đơn theo tầng}
      begin
        add(0,tree);
        dec(long);{không che độ dài hợp lệ}
      end;
  until false;
end;

procedure find(floor:byte; p:r);{sắp ra 20 tân suất lớn nhất}
var i,j:byte;{biên lặp}

```

```

begin
  if floor>=a{do dai nam trong yeu cau} then
    begin
      {sap chen}
      j:=now;
      while (j>0) and (t[j]<p^.num) do dec(j);

  if j<n then
    begin
      if (j=0) or (t[j]>p^.num) then
        begin
          if now<n then inc(now);
          for i:=now-1 downto j+1 do t[i+1]:=t[i];
          t[j+1]:=p^.num;
        end;

        if floor<b then{tiep tuc tim}
          begin
            find(floor+1,p^.zero);
            find(floor+1,p^.one);
          end;
        end;
      else
        begin{tiep tuc tim}
          find(floor+1,p^.zero);
          find(floor+1,p^.one);
        end;
      end;
    end;

procedure look_up(floor:byte; p:r; ss:ar){xuat xau con theo thu tu}
begin
  if (floor=where) and (p^.num=t[which]) then{du tang va tan suat thi in}
    write(outf,' ',ss);

  if (floor<where) and (p^.num>=t[which]) then{tiep tuc di xuong}
    begin
      {cung tan suat va do dai thi xuat theo gia tri giam dan}
      look_up(floor+1,p^.one,ss+'1');
      look_up(floor+1,p^.zero,ss+'0');
    end;
  end;

procedure print;{xuat phuong an cu the}
begin
  now:=0;
  fillchar(t,sizeof(t),0);
  find(0,tree);{sap ra 20 tan suat lon nhat}

```

```

assign(outf,outfile);
rewrite(outf);
settextbuf(outf,tch);
for which:=1 to n do{theo thu tu tan suat giam dan}
begin
  write(outf,t[which]);
  for where:=b downto a do{cung tan suat thi xau dai hon xuat truoc}
  begin
    look_up(1,tree^.one,'1');
    look_up(1,tree^.zero,'0');
  end;
  writeln(outf);
end;
close(outf);
end;

begin
assign(inf,inputfile);
reset(inf);
settextbuf(inf,tch);
init;{khởi tạo chương trình}
main;{đọc dữ liệu, công số lần xuất hiện của xâu con}
close(inf);
print;{xuất phương án cụ thể}
end.

```

Chương trình 3

```

{$A+,B-,D+,E+,F-,G-,I+,L+,N-,O-,P-,Q-,R-,S-,T-,V+,X+}
{$M 65520,0,655360}
const inputfile = 'contact.in';
      outfile = 'contact.out';

      tc :array[1..13] of integer=
        (1,2,4,8,16,32,64,128,256,512,1024,2048,4096);
      {các số nhị phân tương ứng là 1,10,100,..., tiến cho lấy bit}

      put:array[1..12] of integer=
        (1,3,7,15,31,63,127,255,511,1023,2047,4095);
      {các số nhị phân 1,11,111,..., tiến cho lấy 1,2,3,... bit cuối}
      tz:array['0'..'1'] of byte=(0,1);{chuyển ký tự thành số}

type ar=array[0..4095] of longint;

var t :array[1..21] of longint;{20 tần suất lớn nhất, thêm 0 21 để sắp}
    tk :array[1..12] of ^ar;{lưu số lần xuất hiện của từng loại xâu con}
    tch:array[1..1024] of char;{bộ đệm}

```

```

inf,outf:text;
a,b,n{bien vao cua de},long{do dai xau con hop le hien tai},
kind{so tan suat khac nhau hien co khi sap}:byte;
yu:integer;{moc cat do dai hop le}

procedure init;{khởi tạo chương trình}
var i:byte;{biên lặp}
begin
  readln(inf,a,b,n);{nhập a,b,n để khởi tạo}
  for i:=1 to b do{khởi tạo mảng 0}
  begin
    new(tk[i]);
    fillchar(tk[i]^,sizeof(tk[i]^),0);
  end;
end;

procedure main;{đọc dữ liệu, cộng số lần xuất hiện của xâu con}
var i:byte;{biên lặp}
    ch:char;{ký tự hiện đọc}
    m:integer;{biểu diễn của xâu con hợp lệ}
begin
  m:=0;{xâu hợp lệ ban đầu rỗng} long:=0;{chưa đọc dữ liệu}
  yu:=tc[b+1];{ví dụ độ dài hợp lệ 2 thì lấy phần dư theo 4}
  repeat
    read(inf,ch);
    if ch='2' then exit;{đã đọc hết tệp}
    m:=(m*2+tz[ch]) mod yu;{sinh xâu con hợp lệ hiện tại}
    if long<b then inc(long);{tính độ dài xâu con hợp lệ hiện tại}

    for i:=1 to long do inc(tk[i]^[m and put[i]]);
    {cộng các xâu con hợp lệ kết thúc tại ký tự hiện đọc}
  until false;
end;

procedure sort;{sắp ra 20 tần suất lớn nhất}
var i,j,k,ii:integer;{biên lặp}
begin
  fillchar(t,sizeof(t),0);
  kind:=0;
  {tìm tất cả xâu con có thể có}
  for i:=a to b do
    for j:=0 to put[i] do
      begin
        k:=kind;
        {sắp chen}
        while (k>0) and (tk[i]^j>t[k]) do
          dec(k);

```

```

if (k<n) and ((k=0) or (tk[i]^j<>t[k])) then
begin
  if kind<n then inc(kind);

  for ii:=kind downto k+1 do
    t[ii+1]:=t[ii];
  t[k+1]:=tk[i]^j;
end;
end;
end;

procedure print;{xuat ket qua theo yeu cau}
var ii,i,j,l{bien lap},p{bien trung gian de sinh phuong an}:integer;
begin
  assign(outf,outfile);
  rewrite(outf);
  settextbuf(outf,tch);
  for ii:=1 to n do{theo thu tu tan suat giam dan}
  begin
    write(outf,t[ii]);
    for i:=b downto a do{cung tan suat thi do dai giam dan}
      for j:=put[i] downto 0 do{cung do dai thi gia tri giam dan}
        if tk[i]^j=t[ii] then
          begin
            write(outf,' ');
            p:=j;
            for l:=1 to i do{in xau con cu the}
              begin
                write(outf,p div tc[i+1-l]);
                p:=p mod tc[i+1-l];
              end;
            end;
          writeln(outf);
        end;
      close(outf);
    end;

begin
  assign(inf,inputfile);
  reset(inf);
  settextbuf(inf,tch);
  init;{khởi tạo chương trình}
  main;{đọc dữ liệu, công số lần xuất hiện của xâu con}
  close(inf);
  sort;{sắp ra 20 tan suat lon nhat}
  print;{xuat ket qua theo yeu cau}
end.

```


Chương trình 4

```
{ $A+,B-,D+,E+,F-,G-,I+,L+,N-,O-,P-,Q-,R-,S-,T-,V+,X+}  
{ $M 65520,0,655360}
```

```
const inputfile = 'words.inp';  
    textfile = 'text.inp';  
    outfile = 'codes.out';  
  
type rrr={du lieu luu cho quy hoach dong}  
    record  
        num:byte;{so hieu ma tu}  
        up{ma tu truoc}:integer;  
        tot{tong do dai ma tu lon nhat hien tai},  
        start,finish{vi tri bat dau va ket thuc}:longint;  
    end;  
    ar=array[1..100] of longint;{luu tinh hinh khop giua ma tu va van ban}  
    aaa=^aar;  
    aar=record{tinh hinh xuat hien cua ky tu trong cac ma tu}  
        num{so hieu ma tu},ci{vi tri cua ky tu trong ma tu}:byte;  
        next:aaa;  
    end;  
  
var inf,outf:text;  
    nn{so ma an},n{so ma tu},  
    findwz{vi tri cuoi cung xuat hien gia tri lon nhat}:integer;  
    m{vi tri van ban hien doc},findmax{tong do dai ma tu lon nhat}:longint;  
    tn:array[1..10000] of ^rrr;  
    start:array[1..100] of ^ar;  
    tl:array[1..100] of byte;{do dai tung ma tu}  
  
    tv:array[1..1024] of char;  
    t:array['A'..'z'] of aaa;  
    pp:aaa;  
  
procedure put_in(cch:char; nnum{so hieu ma tu},nci:byte{vi tri xuat hien});  
begin{ghi lai vi tri cua ky tu}  
    new(pp);  
    pp^.next:=t[cch]^next;  
    t[cch]^next:=pp;  
  
    with pp^ do  
    begin  
        num:=nnum;  
        ci:=nci;  
    end;  
end;
```

```

procedure init;{doc tep ma tu, khoi tao}
var i,j:byte;{bien lap}
    ii:char;{bien lap}
    s:string;{bien trung gian}
begin
    assign(inf,inputfile);
    reset(inf);
    setttextbuf(inf,tv);
    readln(inf,n);
    for ii:='A' to 'z' do{khoi tao mang t}
        if ii in ['A'..'Z','a'..'z'] then
            begin
                new(t[ii]);
                t[ii]^next:=nil;
            end;

    for i:=1 to n do{khoi tao va luu du lieu}
        begin
            new(start[i]);
            fillchar(start[i]^,sizeof(start[i]^),0);
            readln(inf,s);
            tl[i]:=ord(s[0]);
            for j:=1 to tl[i] do{ghi lai vi tri cua ky tu}
                put_in(s[j],i,j);
            end;
        close(inf);
    end;

procedure clear(snum:byte);{xoa cac gia tri bang nhau}
var i:byte;
    now:longint{bien trung gian};
begin
    now:=start[snum]^tl[snum];
    start[snum]^tl[snum]]:=0;
    for i:=tl[snum]-1 downto 1 do
        if start[snum]^i<>now then exit
        else
            start[snum]^i:=0;
    end;

procedure cl(sstart:longint; snum:byte);{quy hoach dong cho gia tri hien tai}
var i,j,k,maxwz{vi tri gia tri lon nhat truoc ma an hien tai}:integer;
    maxtot{gia tri lon nhat truoc ma an hien tai},
    pnow{gia tri muc tieu hien tai}:longint;
    b:boolean;{dau hieu logic}
begin
    maxtot:=0; maxwz:=0;
    i:=1;

```

```

while (i<=nn) and (tn[i]^finish<sstart) do{tim gia tri lon nhat truoc do}
begin
  if tn[i]^tot>maxtot then{kiem tra va thay gia tri lon nhat hien tai}
  begin
    maxtot:=tn[i]^tot;
    maxwz:=i;
  end;
  inc(i);
end;

pnow:=maxtot+tl[snum]; {tinh gia tri khop lon nhat}
b:=false;
if (nn=0) or (tn[nn]^finish<m) then
{dua ma an hien tai vao mang}
begin
  inc(nn);
  new(tn[nn]);
  b:=true;
end
else
  if pnow>tn[nn]^tot then b:=true;

if b then
begin
  if pnow>findmax then{ghi gia tri muc tieu lon nhat hien tai}
  begin
    findmax:=pnow;
    findwz:=nn;
  end;

  with tn[nn]^ do{gia tri lon nhat ket thuc bang ma tu vua tim}
  begin
    up:=maxwz; num:=snum;
    tot:=pnow;
    start:=sstart;
    finish:=m;
  end;
end;

clear(snum);
{Sau khi tim mot day an phai nho nhat, cac gia tri cu trong mang co the
gay bat loi cho truy hoi ve sau. Vi du van ban TUNN va ma tu TUN: khi tim
duoc TUN, mang hien la 1,1,1. Neu giu lai, ta lai tim thanh ma an TUNN.
Vi vay phai xoa cac vi tri chua tim duoc ma an moi hon, tuc vi tri bat dau
nam sau hon; co the goi thao tac nay la xoa cac gia tri bang nhau.}
end;

procedure main;{doc va xu ly van ban}

```

```

var i:integer;{bien lap}
    nnum,nci{bien trung gian}:byte;
    ch:char;{ky tu hien tai}
begin
    assign(inf,textfile);
    setttextbuf(inf,tv);
    reset(inf);
    m:=0;
    while not eof(inf) do
        begin
            read(inf,ch);
            pp:=t[ch];
            inc(m);
            while pp^.next<>nil do{lan luot tim vi tri cua ky tu}
                begin
                    pp:=pp^.next;
                    nnum:=pp^.num; nci:=pp^.ci;
                    if nci=1 then{xu ly theo do dai va vi tri hien khop}
                        if tl[nnum]=1 then cl(m,nnum)
                        else
                            start[nnum]^[1]:=m
                        else
                            if (start[nnum]^[nci-1]<>0) and
                                (m-start[nnum]^[nci-1]+1<=1000){dieu kien bien} then
                                    if nci=tl[nnum] then cl(start[nnum]^[nci-1],nnum)
                                    else
                                        start[nnum]^[nci]:=start[nnum]^[nci-1];
                            end;
                        end;
                    end;
                close(inf);
            end;

procedure print;{xuat ket qua}
var k:integer;{bien truy nguoc}
begin
    assign(outf,outfile);
    rewrite(outf);
    setttextbuf(outf,tv);
    writeln(outf,findmax);
    k:=findwz;

    while k<>0 do{truy nguoc tim phuong an cu the}
        begin
            writeln(outf,tn[k]^num,' ',tn[k]^start,' ',tn[k]^finish);
            k:=tn[k]^up;
        end;
    close(outf);
end;

```

```
begin
nn:=0; findmax:=0;
init;{doc tep ma tu, khoi tao}
main;{doc va xu ly van ban}
print;{xuat ket qua}
end.
```

C Tài liệu tham khảo

References

- [1] Yan Weimin, Wu Weimin. *Cấu trúc dữ liệu*, tái bản lần thứ hai. Nhà xuất bản Đại học Thanh Hoa.
- [2] Tổ huấn luyện đội tuyển IOI Trung Quốc. *Tuyển tập luận văn xuất sắc của đội tuyển tập huấn Trung Quốc IOI 1998*.
- [3] Wang Fan, Ni Zhaozhong, chủ biên. *Olympic Tin học*, quý san.
- [4] Wu Wenhui, Wang Jiande. *Hướng dẫn thi Olympic Tin học quốc tế và toàn quốc cho thanh thiếu niên: thuật toán và thiết kế chương trình về đồ thị*. Nhà xuất bản Đại học Thanh Hoa.